

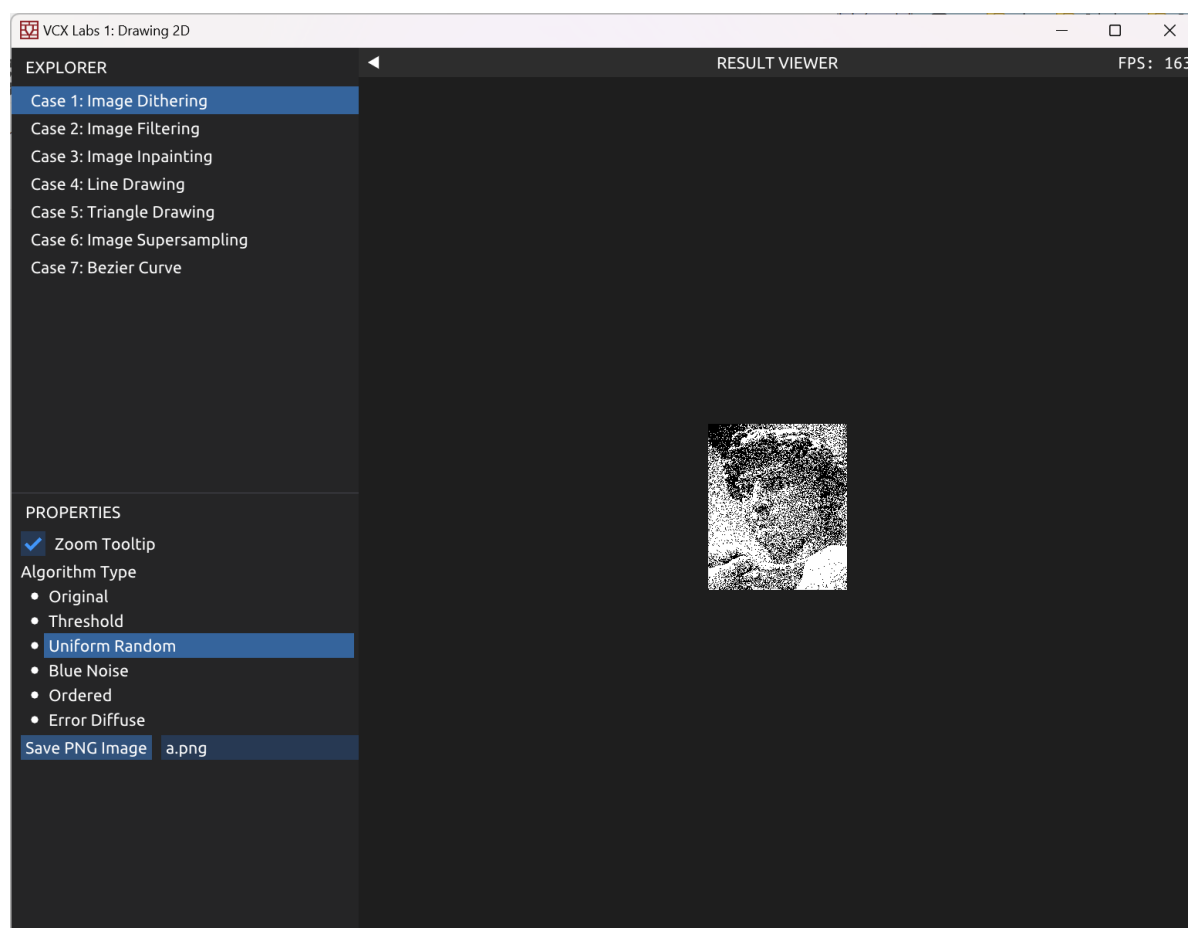
Lab1实验报告

蒲彦丞-2200012956

Task 1: Image Dithering

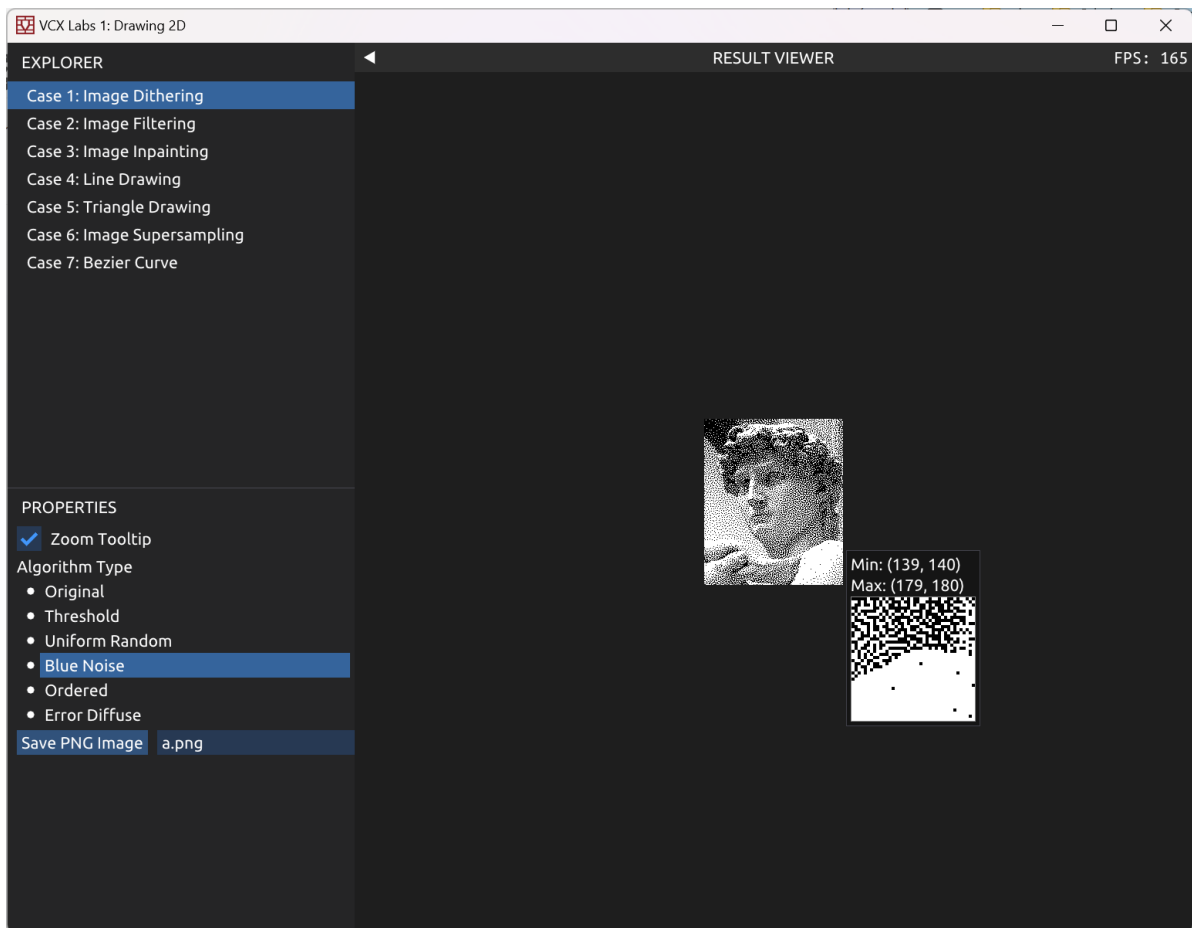
Uniform Random

按照要求来就好，结果如图：



Blue Noise Random

最开始没有意识到 **Threshold** 的界应设为1。加上蓝噪声后每个店灰度值取值范围为 $[0, 2]$ ，故 **Threshold** 的界应设为1，最终结果如图：

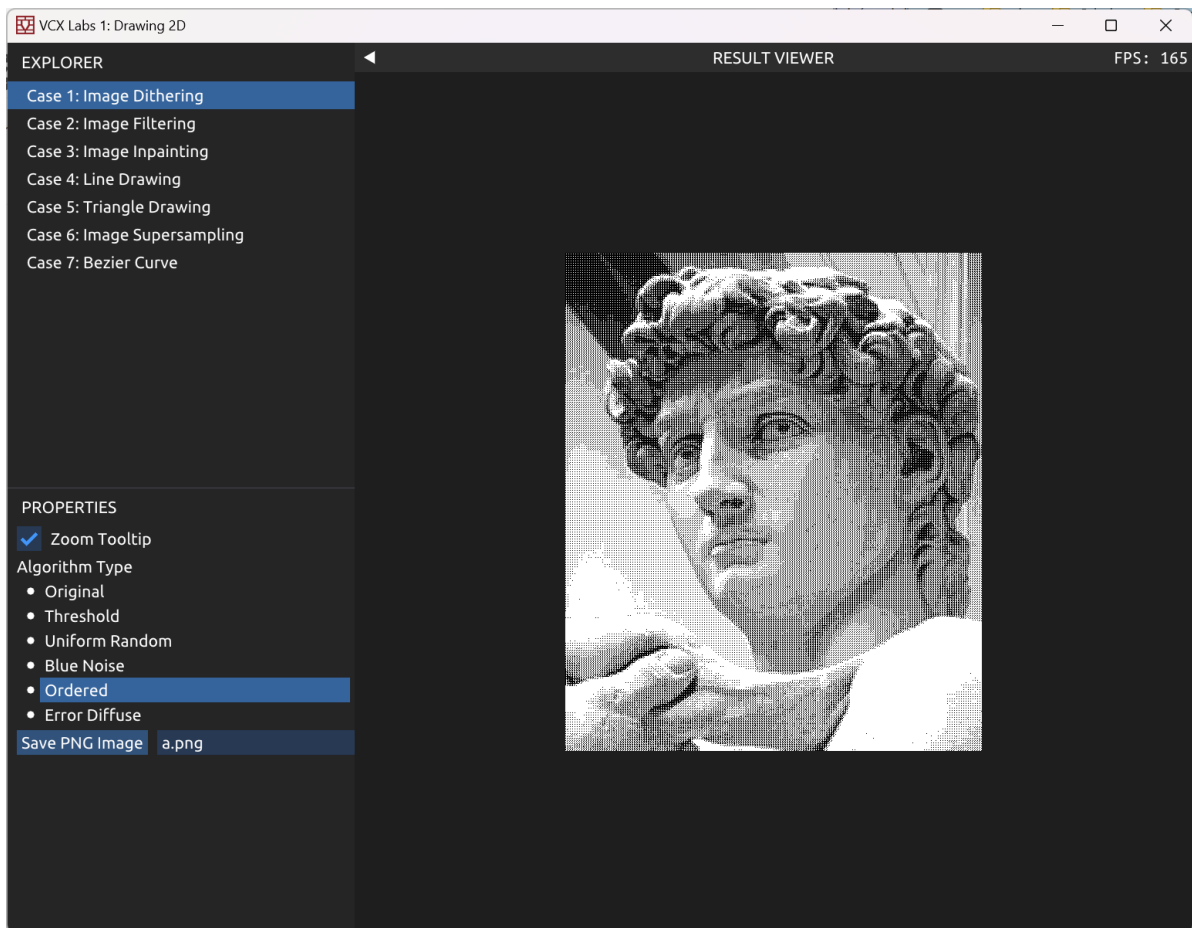


Ordered

在本任务中我使用矩阵为讲义中矩阵

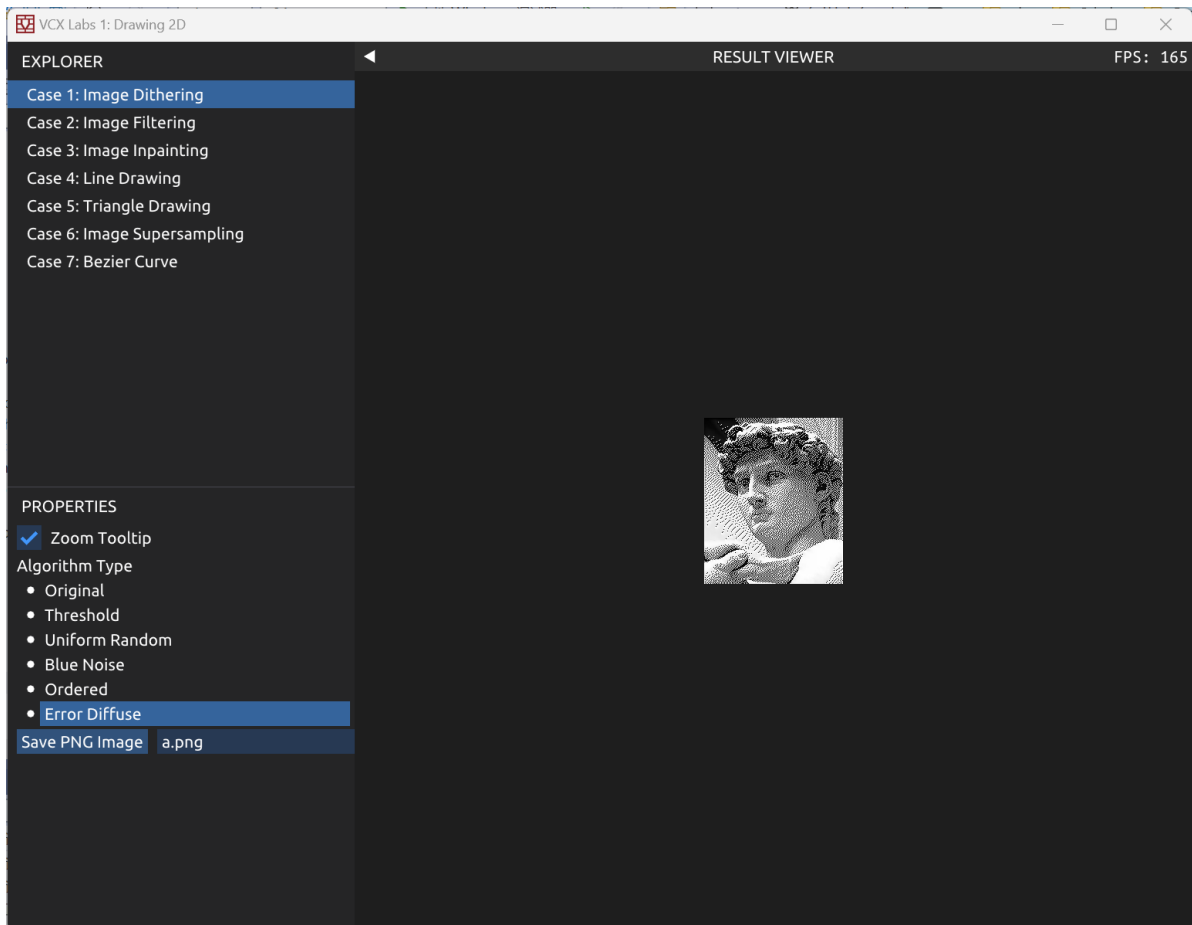
$$\begin{pmatrix} 6 & 8 & 4 \\ 1 & 0 & 3 \\ 5 & 2 & 7 \end{pmatrix}$$

最终实现效果如图:



Error Diffuse

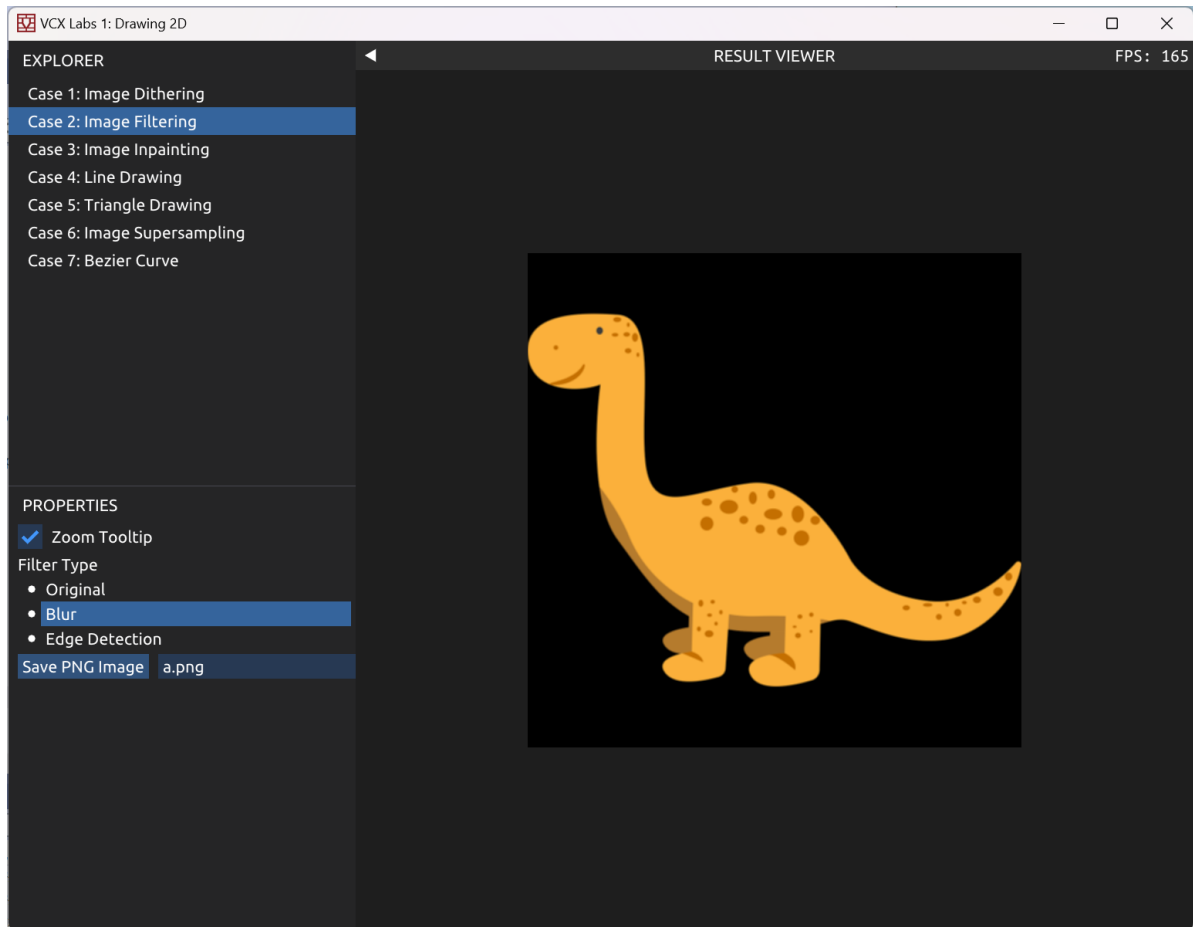
注意边界处理即可，效果如图：



Task 2: Image Filtering

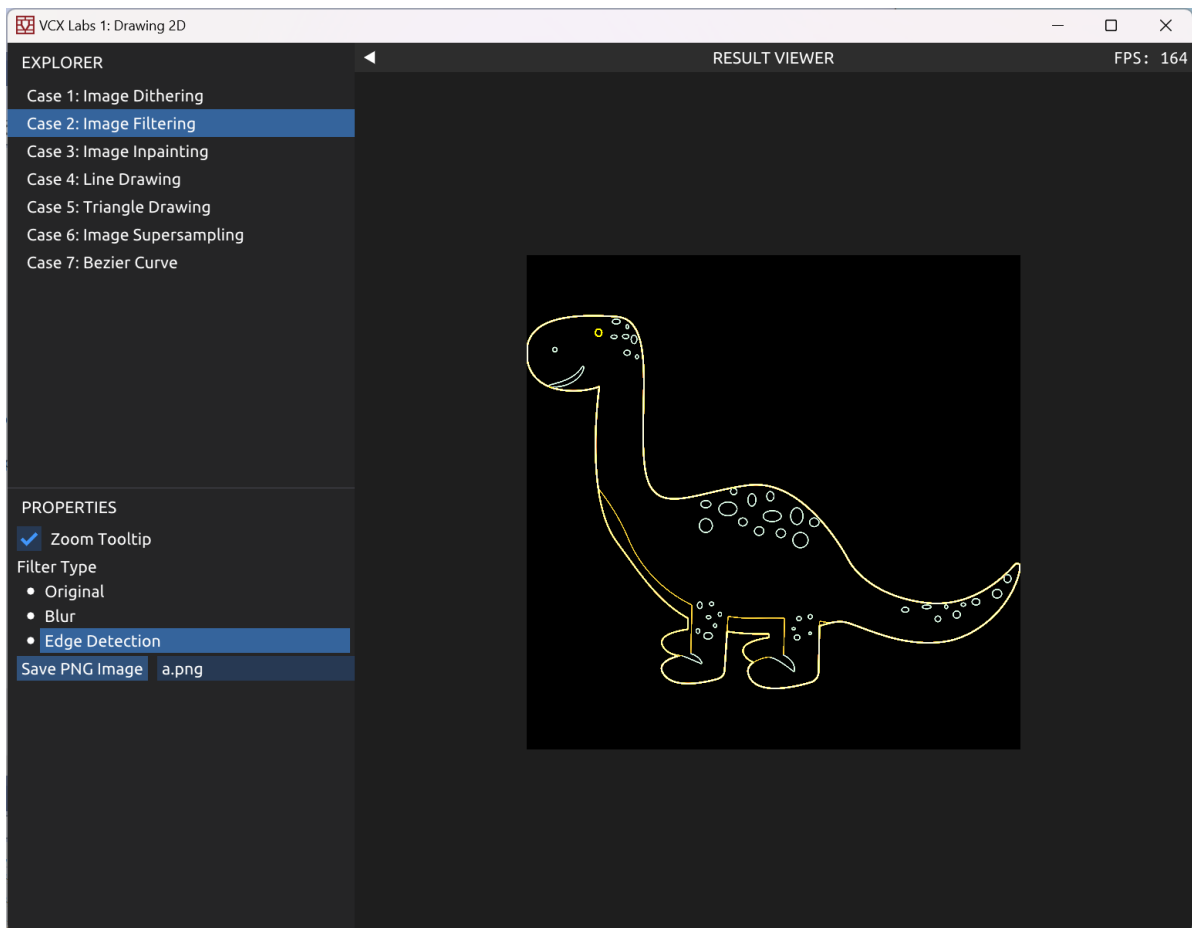
图像模糊:

当时写的时候以为不能定义函数或者加头文件，直接当 `if` 仙人了，实现效果如图：



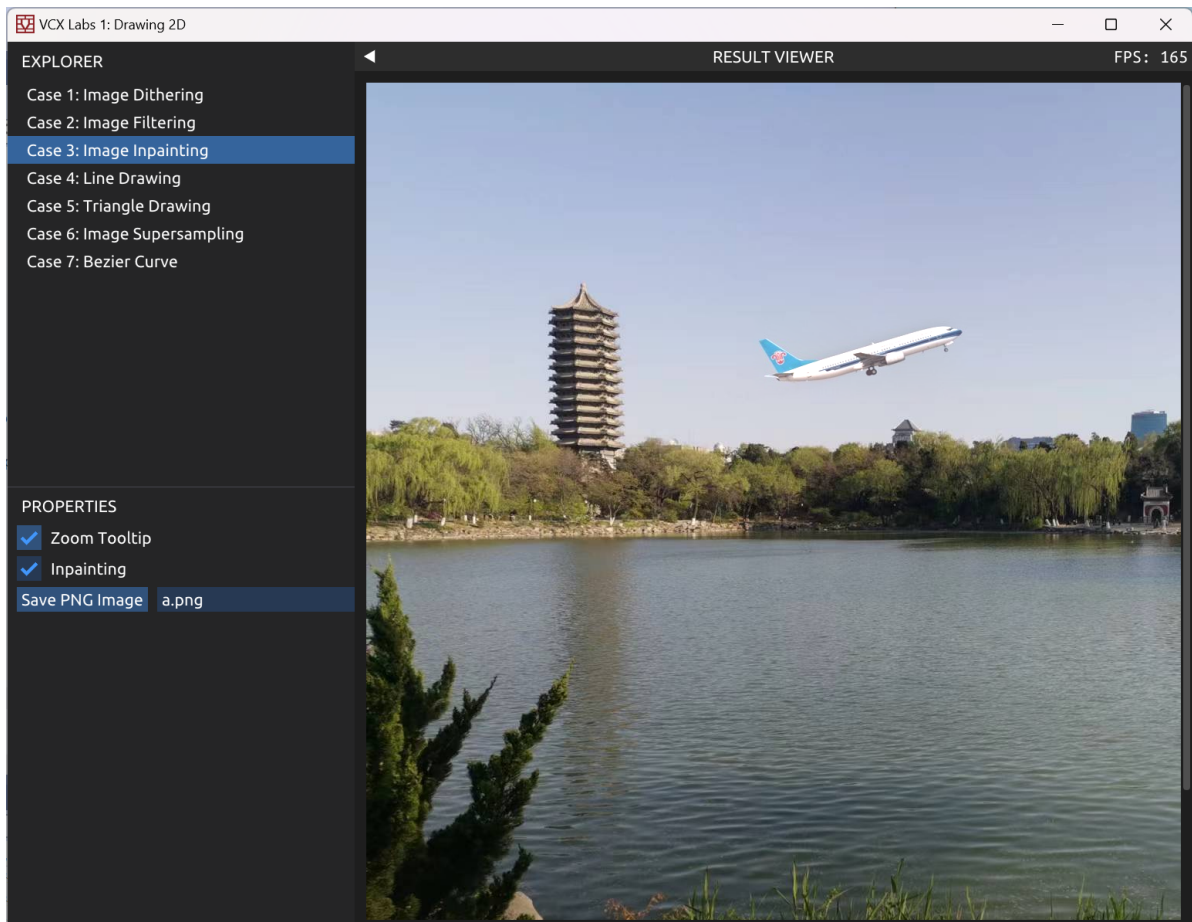
图像边缘提取:

按照三颜色通道分别提取 `if (r_magnitude + g_magnitude + b_magnitude > 1.5)` 则保留为边界，结果如图：



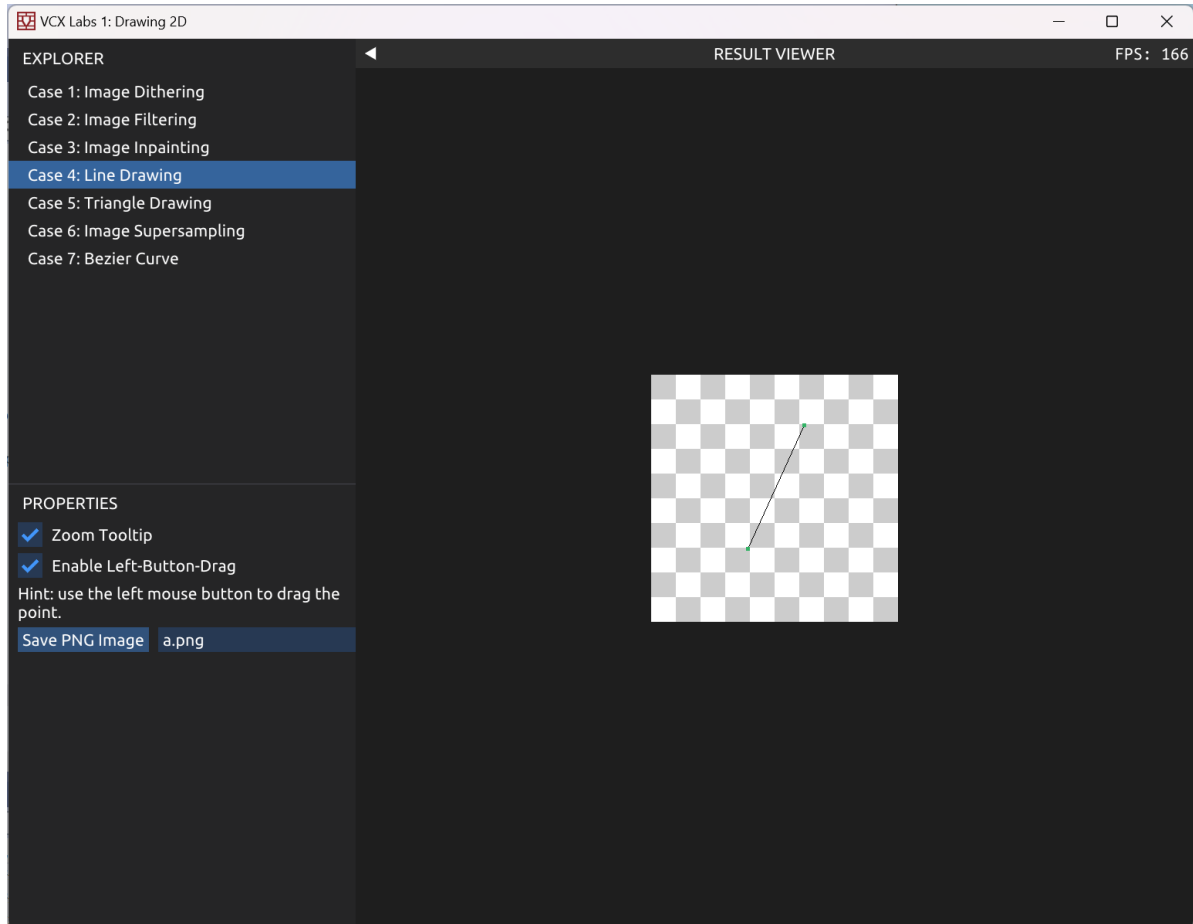
Task 3: Image Inpainting

阅读代码发现最后一步加上了 `inputFront` 故 `g` 边界应该为 `inputBack - inputFront` , 实现结果如图:



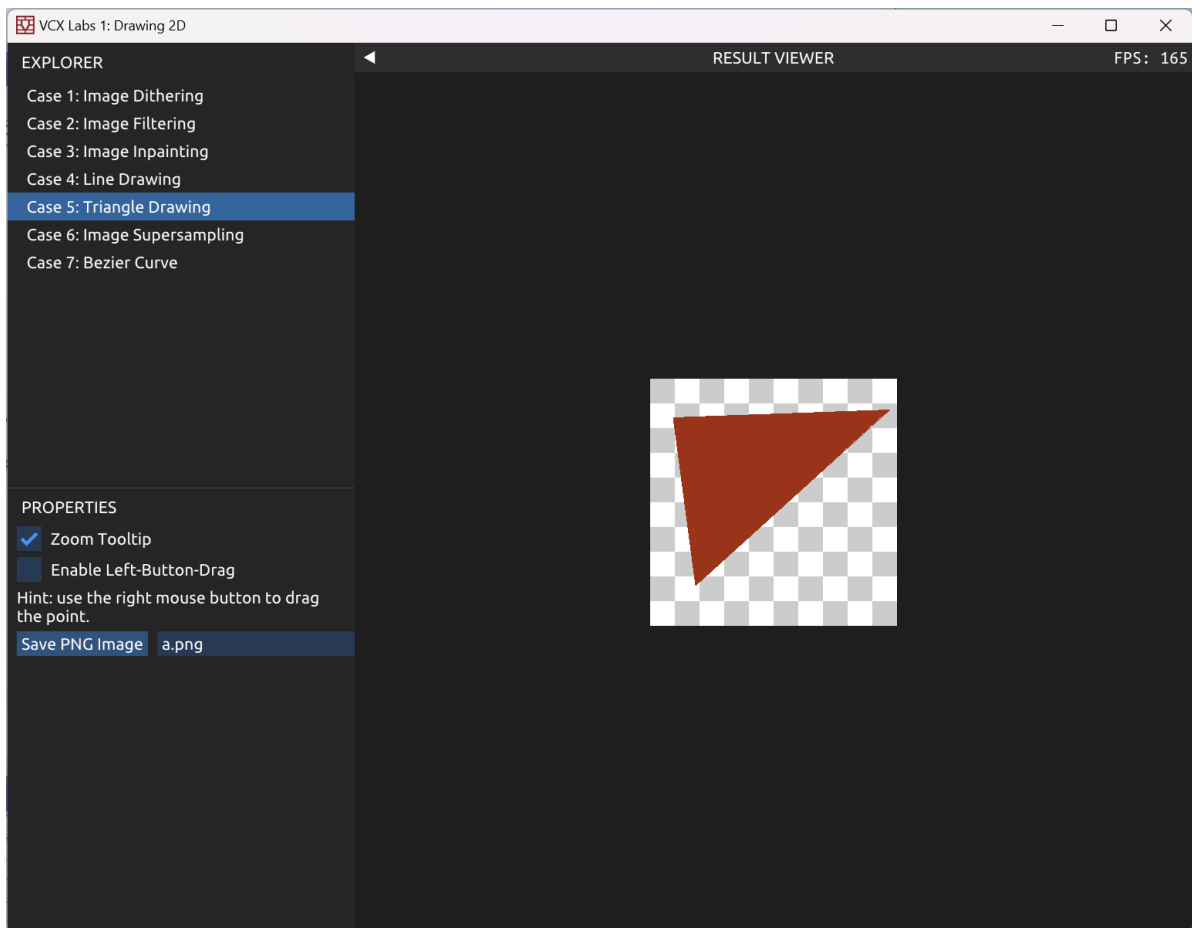
Task 4: Line Drawing

用 *Bresenham* 算法绘制直线,实现结果如图:



Task 5: Triangle Drawing

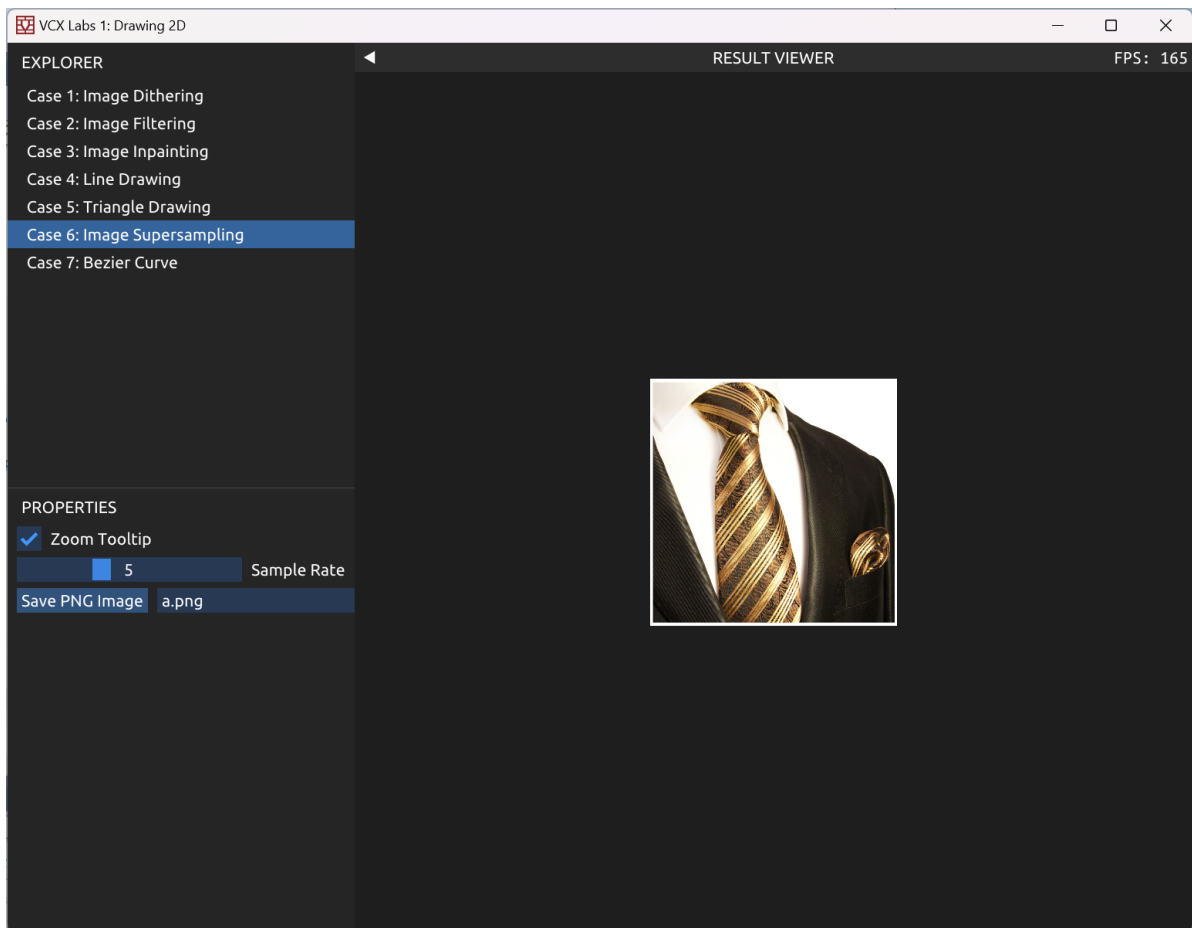
参照课件中的伪代码实现即可,想法是相对点按 y 大小排列,随后分两端进行水平划线即可。过程中可以采用 *Bresenham* 算法的思想维护两边的断点以提高效率.实现结果如图:



Task 6: Image Supersampling

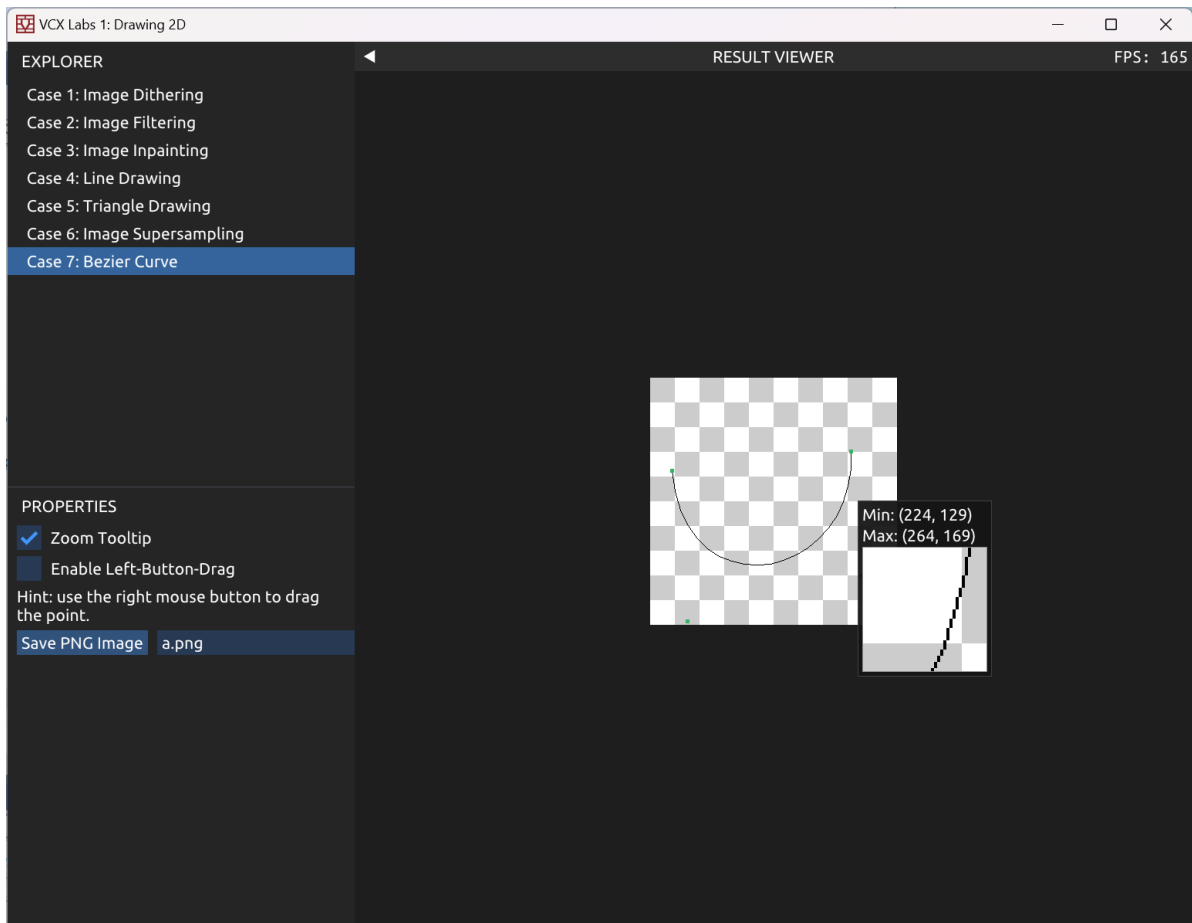
最初误解了题目意思，以为是缩放图片。

处理思路为找到 `input` 与 `output` 对应点计算周围 `rate * rate` 个点的平均即可。这里稍微偷懒对边缘点也是按照 `rate * rate` 做均值,实现结果如图:



Task 7: Bezier Curve

实现德卡斯特里奥算法，结果如图：



小结:

lab1到此结束，总体还是比较好做，主要花费时间在阅读理解 `ImageRGB` 等的定义上，还有就是理解 *Poisson* 算法用了一些时间（说到底数值分析不是大二下的课吗，汗流浹背了）